KYNARA

Enterprise Setup Guide

A production onboarding guide for governing AI agents with Kynara — identity, policies, enforcement, approvals, and compliance.

Kynara · v2.3 · June 2026

CONTENTS

1 Introduction

Kynara is the permission control plane for AI agents. It decides — in real time, before an agent acts — what each agent is allowed to do, on whose behalf, and under what conditions, then records every decision in a tamper-evident audit log. This guide walks an enterprise team through a production setup end to end.

Who this is for: platform, security, and IAM teams deploying Kynara to govern AI agents built on LangChain, CrewAI, AutoGen, OpenAI, Anthropic, or MCP servers.

- **Identity & access** — agents, roles, scopes, and Okta agent-identity sync.
- **Policies** — RBAC + ABAC, including argument-level allowlists (Slack/Gmail).
- **Enforcement** — SDKs and the MCP Gateway.
- **Oversight** — approvals, tamper-evident audit, and compliance.

2 Architecture overview

Kynara sits **outside the LLM's trust boundary**. Your agent (or a gateway in front of it) calls a decision API with structured data — subject, action, resource, context — and gets back `allow`, `deny`, or `require_approval`. Because it evaluates structured requests rather than natural language, a prompt injection inside the model cannot change what Kynara receives or how it decides.

- **Decision engine** — evaluates RBAC (role grants) then ABAC (policy conditions).
- **Non-escalation** — an agent's effective permissions are the intersection of the agent's and the dispatching user's roles.
- **Fail-closed** — if the control plane is unreachable, the SDKs deny by default.
- **Tamper-evident audit** — every decision is appended to a SHA-256 hash-chained log.

> **NOTE** Enforcement happens at the tool boundary, so a `deny` means the side effect never runs.

3 Prerequisites

- A Kynara organization (cloud signup at kynaraai.com) or a self-hosted deployment.
- An owner/admin seat to configure the org.
- For SSO: your IdP's SAML/OIDC metadata. For agent sync: an Okta API token.
- For SDK enforcement: Python 3.10+ or Node 18+. For MCP: your upstream MCP server URL.

4 Deployment options

Kynara Cloud

Sign up, create your organization, and you're ready. Kynara Cloud is multi-tenant with per-org isolation (row-level security).

Self-host

Kynara is source-available (BSL 1.1). Run the full stack in your own environment with Docker/Helm. The backend container runs database migrations automatically on start (`alembic upgrade head`), then serves the API; the frontend is served by nginx. Set `DATABASE_URL`, a strong `JWT_SECRET`, and an encryption key before first boot.

> **IMPORTANT** Always set a strong `JWT_SECRET` and a dedicated encryption key in production. The entrypoint refuses to start with default secrets.

5 Organization & team setup

1. In **Settings** → **Members**, invite your team and assign seat roles: `owner`, `admin`, `developer`, `auditor`, or `member`.
2. In **Settings** → **Identity providers**, configure SSO via SAML 2.0 or OIDC (Okta, Entra, etc.).
3. Enable SCIM provisioning to keep human users in sync with your directory.

Seat role	Typical use
owner / admin	Full configuration: policies, agents, billing, integrations.
developer	Build agents, author policies, run simulations.
auditor	Read-only access to policies, decisions, and the audit log.
member	Day-to-day use without administrative rights.

6 Agent identities

Every agent needs a registered identity before Kynara can govern it. You can create agents manually or sync them from your IdP.

Register manually

Agents → **New agent**: set a display name, supervision mode (`human_supervised`, `autonomous`, or `read_only`), and a daily action budget. Copy the Agent ID — your SDK passes it as `subject_id`.

Sync from Okta (Identity Providers)

1. Go to **Administration** → **Identity Providers** → **Connect Okta**.
2. Enter your Okta org URL and an SSWS API token (stored encrypted).
3. Choose a sync mode: **Agents** (Okta's agent identities) or **Group** (members of a designated group).
4. Optionally map Okta groups to Kynara roles, and pick the user synced agents act on behalf of.
5. Click **Test**, then **Sync now**. Sync is idempotent — re-running updates rather than duplicates.

> **NOTE** Okta owns identity; Kynara owns authorization. Imported agents become standard Kynara agents, so all policies, approvals, and audit apply automatically.

7 Roles & scopes (RBAC)

Roles grant **scopes** (capabilities). An agent can only perform an action if a role it holds grants a matching scope — this is the first gate, evaluated before any policy.

1. In **Access Control** → **Roles**, create a role (e.g. `crm-reader`) and add the scopes it needs (e.g. `crm:contacts.read`). Wildcards like `crm:*` are supported.
2. In **Agents** → **[agent]** → **Roles**, assign the role to the agent, bound to the user it acts on behalf of.
3. Confirm the agent's **Access Summary** now lists the granted scopes.

> **IMPORTANT** Scope strings are matched literally. A role granting `crm.contacts.read` (dots) will not satisfy an action of `crm:contacts.read` (colon). Keep notation consistent across roles, policy actions, and SDK calls.

8 Policies (RBAC + ABAC)

Policies decide the outcome once the RBAC gate passes. Each policy has an effect (`allow` / `deny` / `require_approval`), a priority (lower = evaluated first; first match wins), optional action scopes and resource types, and an optional condition.

Conditions are JSON expressions over `ctx.subject`, `ctx.action`, `ctx.resource`, and `ctx.context`. Operators: `and, or, not, eq, neq, gt, gte, lt, lte, in, contains, starts_with, ends_with, time_between, has_scope`.

Argument-level policies (allowlists)

Conditions can read a tool call's arguments via `ctx.resource.attrs.<key>` — so you enforce *intent*, not just the action. Ready-made templates ship in the policy editor (*Load example*).

```
Slack — only post to approved channels:
{ "op":"in", "args":["ctx.resource.attrs.channel", ["C0123","C0456"]] }

Gmail — only email internal recipients:
{ "op":"ends_with", "args":["ctx.resource.attrs.recipient", "@yourcompany.com"] }

External recipient -> require approval (set effect = require_approval):
{ "op":"not",
  "args":[ {"op":"ends_with","args":["ctx.resource.attrs.recipient","@yourcompany.com"]} ] }
```

> **TIP** Use the **Simulator** (on any policy) to dry-run a subject + action + context and confirm the outcome before deploying. Use **Policy Replay** to see how a change would affect real historical decisions.

9 Enforcement: SDK & MCP Gateway

SDK (Python / TypeScript)

```
pip install kynara-sdk

from kynara_sdk import Kynara, permission_required
from kynara_sdk.context import set_current_kynara

set_current_kynara(Kynara(api_key=KEY, agent_id="crm-assistant",
                          user_id=user, fail_closed=True))

@permission_required("crm.contacts.read", resource_arg="contact_id")
def read_contact(contact_id): return crm.get(contact_id)
```

On `deny`, `PermissionDenied` is raised before the function body runs. On `require_approval`, `ApprovalRequired` is raised so the agent can pause. A `KynaraCallbackHandler` is available for LangChain/LangGraph; AutoGen and CrewAI patterns are documented.

MCP Gateway (no agent code changes)

1. In **Enforcement** → **MCP Gateway**, register your upstream MCP server (URL, scope prefix, fail mode).
2. Run the Kynara MCP wrapper with `KYNARA_MCP_SERVER_ID` and an API key; it discovers the upstream tools and registers them.
3. Each tool auto-maps to a Kynara scope; refine scopes, risk, or pin an effect override per tool.
4. Point your agent at the gateway URL. Every tool call is now authorized per agent, and agents only see tools they're allowed to use (least-privilege discovery).

> **NOTE** Each MCP server has its own fail mode — `closed` denies on engine unreachability (the secure default), `open` allows.

10 Approvals & notifications

When a policy returns `require_approval`, the agent pauses and a request appears in the **Approvals** queue with full context and a risk score. A reviewer approves or rejects; the agent resumes or aborts. Approvals expire after 24 hours by default.

- Configure **Slack** or **Microsoft Teams** under Integrations to review approvals from chat.
- Configure **PagerDuty** to page on-call for critical events (e.g. agent kill-switch).

11 Audit & compliance

Every decision — allow, deny, or require_approval — is appended to a SHA-256 hash-chained, append-only audit log. Any modification of a past record breaks the chain and is detectable via **Verify Chain**. Logs are queryable, filterable, and exportable (CSV).

- Supports **SOC 2, ISO 27001, GDPR, HIPAA** programs; enterprise plans include a HIPAA BAA and custom retention.
- Maps to **EU AI Act Article 12** logging expectations (append-only, traceable, ≥ 6-month retention).

> **TIP** Use the audit log filters and CSV export to produce evidence for a specific agent, action, or time window during an audit or incident.

12 Guardrails & JIT grants

Guardrails auto-revoke an agent that exceeds behavioral thresholds (e.g. too many high-severity events in a window) without waiting for a human — your last line of defense against a runaway or compromised agent.

JIT grants cover break-glass: an operator grants a time-boxed, scoped permission elevation with a justification and optional ticket link. The grant widens the agent's effective scope for its duration, then auto-expires — and is recorded in the audit chain.

13 Security & data handling

- **Authentication:** Argon2id password hashing; short-lived JWTs with rotating refresh tokens; scoped API keys for agents/machines.
- **Secrets:** encrypted at rest (AES via Fernet) — upstream/integration/IdP tokens are never stored in plaintext.
- **Tenancy:** multi-tenant Postgres with row-level security; data residency and BYOK options.
- **Self-host:** source-available (BSL 1.1) so you can run and inspect the full stack.

See the live security page at `kynaraai.com/security` and request security documentation from your account team.

14 Production go-live checklist

- Strong `JWT_SECRET` and encryption key set; secrets stored in a vault, not env files in source.
- Database migrations applied (`alembic upgrade head`) — runs automatically on backend deploy.
- SSO + SCIM configured; seat roles assigned least-privilege.
- Agents registered (or Okta sync enabled); roles grant only required scopes.
- Core policies in place: a low-priority deny baseline, allow rules for legitimate work, approval rules for high-risk actions.
- SDK clients set `fail_closed=True` ; MCP servers set fail mode to `closed` .

- Approval routing (Slack/Teams) and critical alerts (PagerDuty) configured.
- Audit log Verify Chain passes; export tested. Guardrail thresholds set.

15 Support & resources

- Docs: kynaraai.com/docs • Quickstart: kynaraai.com/quickstart
- Security & trust: kynaraai.com/security • Sandbox: kynaraai.com/sandbox
- Testing guide: see the companion *Kynara Complete Testing Guide*.
- Questions or a security review: contact your account team or security@kynaraai.com.